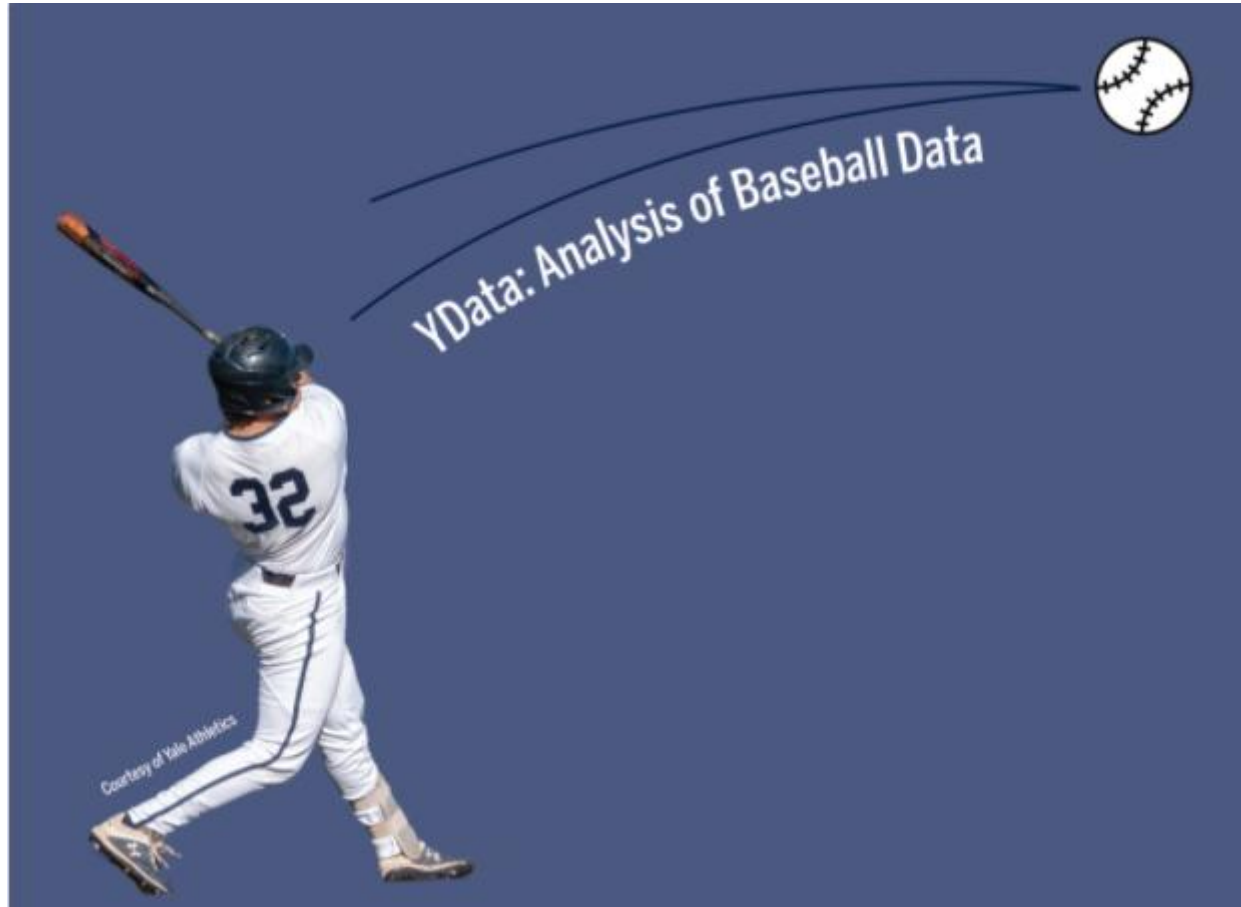


S&DS 1730



Class 11: Regression continued and statistical inference

Overview

Quick review and continuation of linear regression

Regression to the mean, and the Sports Illustrated cover jinx

Quick discussion of Bayesian inference

Polynomial regression

Overfitting

Bill James' Pythagorean Expectation

If there is time: start on lab 9 and/or work on your project

Final projects!

Use the Data Science methods you have learned in this class to answer a question related to baseball!

You will give a 2 minute (one slide) presentation on your project during class on April 20th

- This needs to be turned on by 11pm on Sunday April 19th

Final project written reports are due at 11pm on April 26th

- Report should be a Jupyter notebook approximately 7 pages long
- A final project Jupyter notebook template has been uploaded to the class GitHub site



The baseball season has started!

American League East

Team	W	L	Pct	GB	Home	Away	L10
 Yankees	7	2	.778	-	2-1	5-1	7-2
 Rays	4	5	.444	3.0	0-0	4-5	4-5
 Blue Jays	4	5	.444	3.0	4-2	0-3	4-5
 Orioles	3	6	.333	4.0	3-3	0-3	3-6
 Red Sox	2	7	.222	5.0	1-2	1-5	2-7

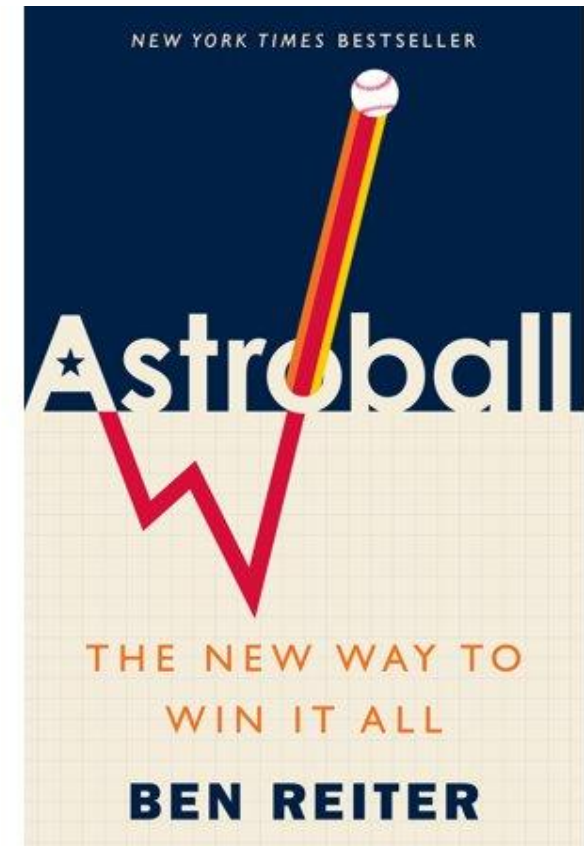
National League West

Team	W	L	Pct	GB	Home	Away	L10
 Dodgers	7	2	.778	-	4-2	3-0	7-2
 Diamondbacks	5	5	.500	2.5	5-2	0-3	5-5
 Padres	4	5	.444	3.0	2-4	2-1	4-5
 Rockies	3	6	.333	4.0	1-2	2-4	3-6
 Giants	3	7	.300	4.5	1-6	2-1	3-7

Discussion on chapter 8 of Astrobball

Let's discuss the chapter 8 for 3 minutes in small groups of 3-4 people and then have a larger conversation as a group

- E.g., Discuss your quote and reaction to chapter 8



Astroball Chapter 8

Trade for Verlander

- Astros pay \$20 million of his salary, Tigers pay \$8 million
 - 2 years on contract



Jake Rogers

Position: Catcher
Bats: Right • Throws: Right
5-11, 201lb (180cm, 91kg)
Team: [Detroit Tigers](#) (majors)

[More bio, uniform, draft, salary info ▼](#)

SUMMARY	WAR	AB	H	HR	BA	R	RBI	SB	OBP	SLG	OPS	OPS+
2026	-	6	1	0	.167	1	1	0	.167	.333	.500	44
Career	2.5	995	199	44	.200	133	130	3	.270	.386	.656	79

34



Daz Cameron

Position: Outfielder
Bats: Right • Throws: Right
6-0, 185lb (183cm, 83kg)
Born: [January 15, 1997](#) (Age: 29-076d) in McDonough, GA

[More bio, uniform, draft, salary info ▼](#)

SUMMARY	WAR	AB	H	HR	BA	R	RBI	SB	OBP	SLG	OPS	OPS+
Career	-1.7	435	87	11	.200	55	42	15	.258	.326	.585	66

41



Franklin Perez

Position: Starting Pitcher
Bats: Right • Throws: Right
6-3, 197lb (190cm, 89kg)
Team: [Detroit Tigers](#) (minors, 40-man)
Born: December 6, 1997 (Age: 23-134d) in Valencia, [Venezuela](#)

Full Name: Franklin Eduardo Perez
[View Player Info](#) from the [B-R Bullpen](#)

Astroball Chapter 8



Kate Upton  @KateUpton · Nov 16, 2016

Hey @MLB I thought I was the only person allowed to fuck @JustinVerlander ?! What 2 writers didn't have him on their ballot?

 3.7K

 76.6K

 115.7K



playerID	yearID	stint	teamID	lgID	W	L	G	GS	CG	SHO	SV	IPouts	H	ER	HR	BB	SO	BAOpp	ERA	IBB	WP	HBP	BK	BFP	GF	R	SH	SF	GIDP
porceri01	2016	1	BOS	AL	22	4	33	33	3	0	0	669	193	78	23	32	189	0.23	3.15	0	3	13	0	890	0	85	2	3	16
verlaju01	2016	1	DET	AL	16	9	34	34	2	0	0	683	171	77	30	57	254	0.207	3.04	1	6	8	0	903	0	81	4	7	8

Astroball Chapter 8

He began tinkering.

He gripped his slider farther back in his hand, and slightly adjusted the position of his wrist, to impart more downward movement on it.

On June 21, he struck out 11 Seattle Mariners in just 5 $\frac{2}{3}$ innings. "I was giddy about it," he said. "I came home and told Kate right away: 'I found it.'"

When Verlander arrived in Houston, the members of the Nerd Cave – especially Mike Fast – had been assiduously analyzing each of his starts, via video and Statcast, for months.

They were dying to ask him one question, though they didn't want to come on too strong. "By the way," they asked, "did you change your grip, or something?". pg 192



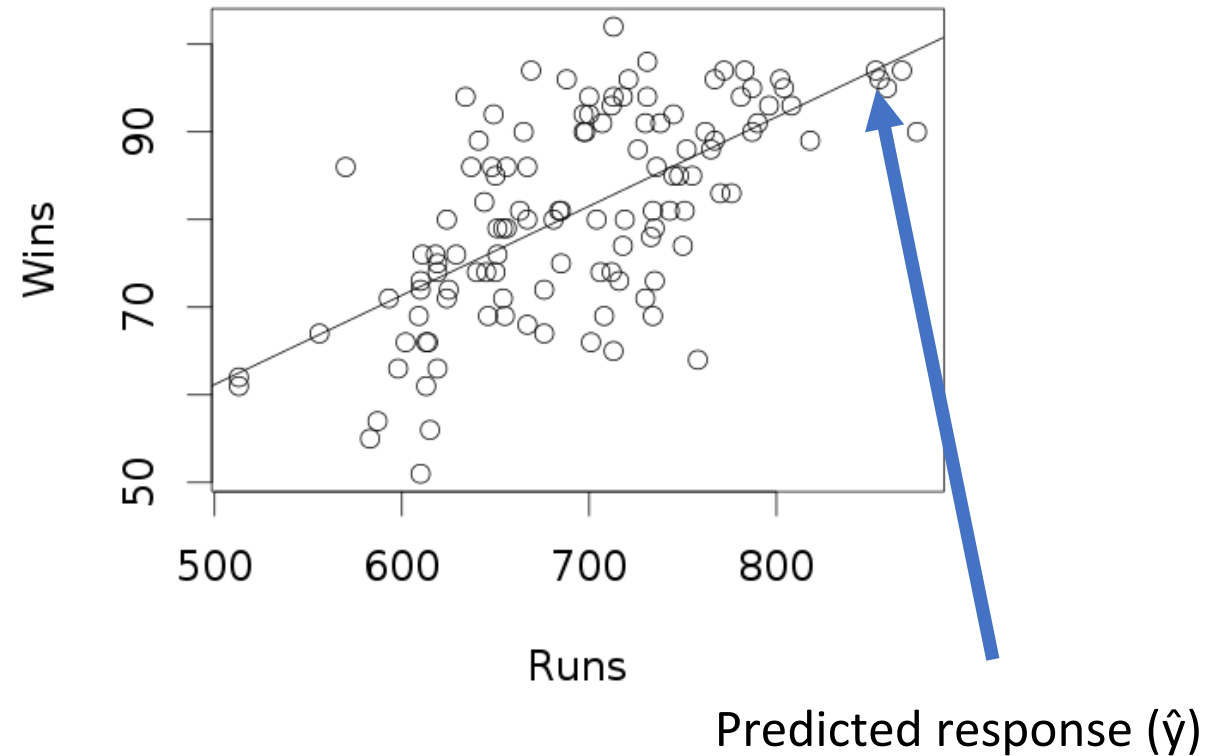
Let's see if we can see the increase in the spin in Verlander's slider in the data!

Quick review and continuation of regression

Regression

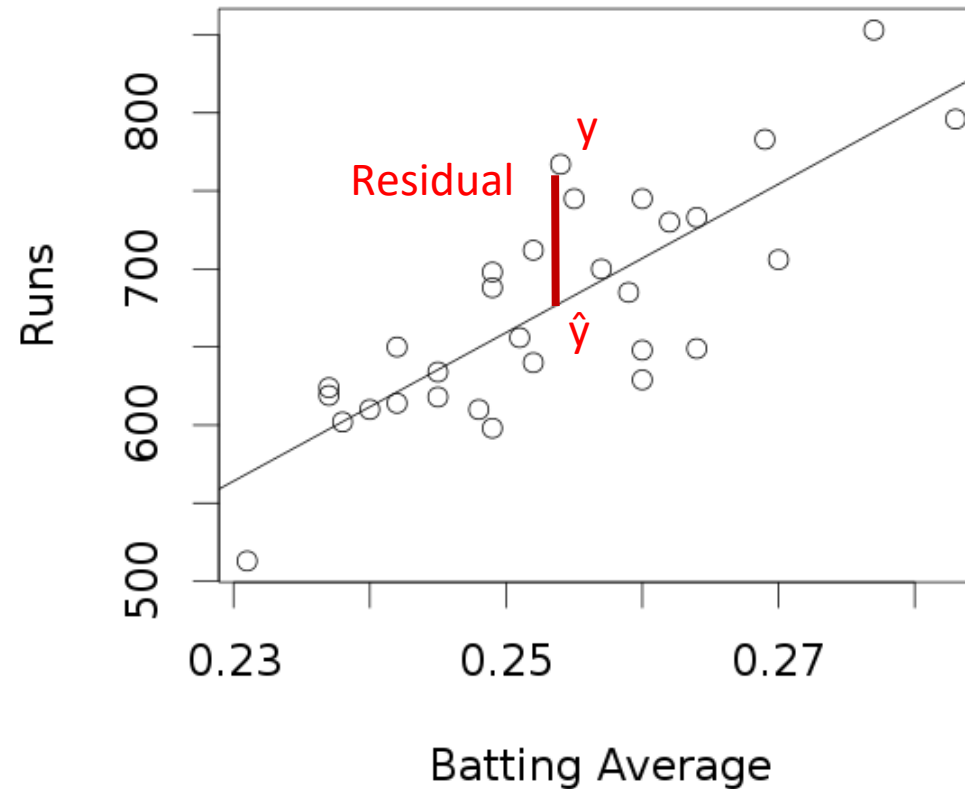
Regression is method of using one variable to predict the value of a second variable

In **linear regression** we fit a line to the data, called the **regression line**



$$\hat{y} = b_0 + b_1 \cdot x$$

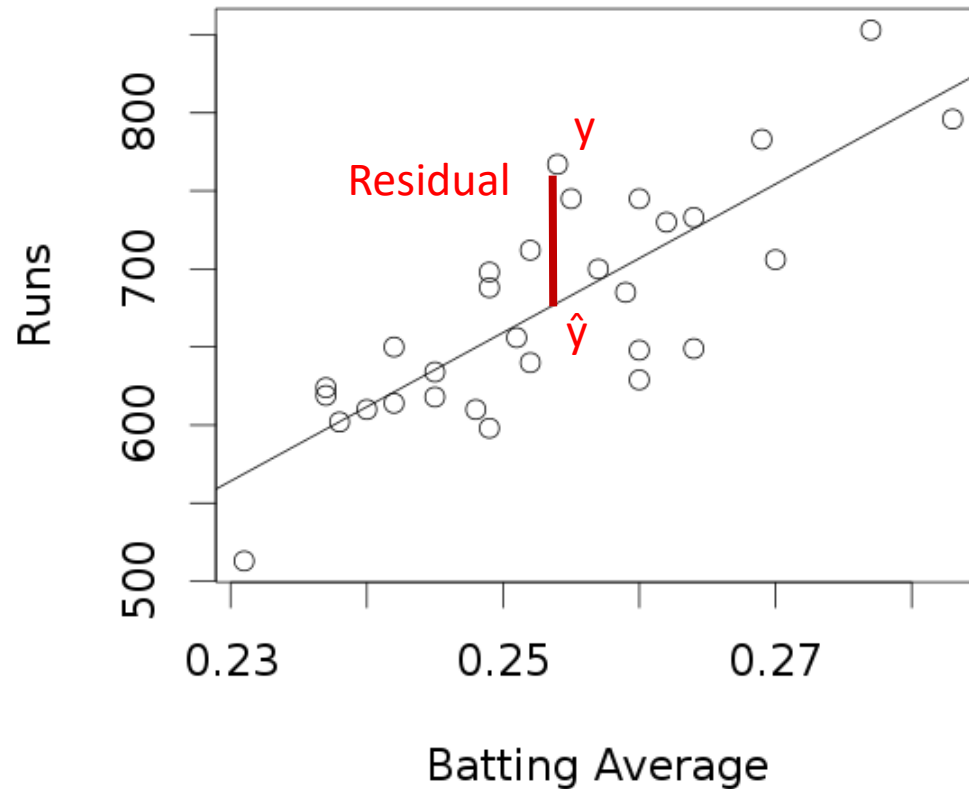
Residuals



The **residual** at a data value is the difference between the observed (y) and predicted value ($\hat{y}$) of the response variable

$$\text{Residual} = \text{Observed} - \text{Predicted}$$

Measuring goodness of fit



$$r^2 = 1 - \text{MSE}/\text{var}(y) \cdot [(n-1)/n]$$

We can measure how well the line fits the data using the equation:

$$MSE = \frac{1}{n} \sum_i^n (y_i - \hat{y}_i)^2 = \frac{1}{n} \sum_i^n (y_i - (b_0 + b_1 \cdot x_i))^2$$

Least squares line

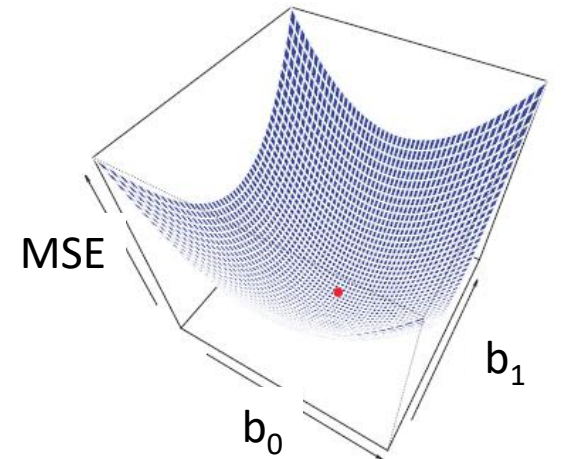
The **least squares line**, also called “**the line of best fit**”, is the line which minimizes the sum of squared residuals

- i.e., the least squares line are the coefficients a , and b that minimize the Mean Squared Error (MSE)

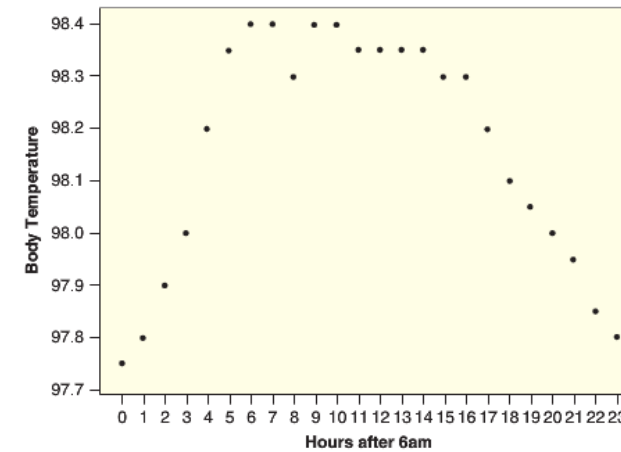
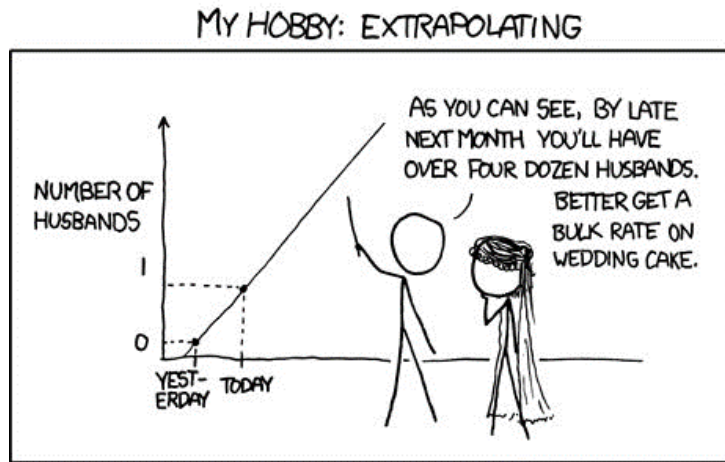
$$MSE = \frac{1}{n} \sum_i^n (y_i - \hat{y}_i)^2$$

The regression coefficients can be found using calculus:

- This can be done by setting the partial derivative of the MSE with respect for b_0 and b_1 to 0 and solving for b_0 and b_1



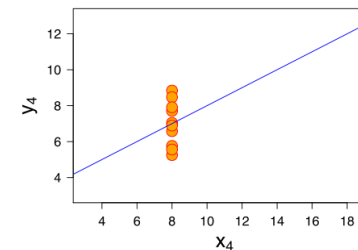
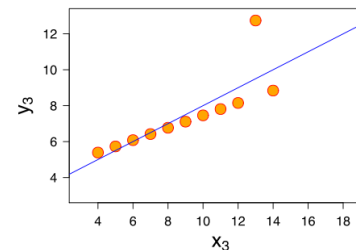
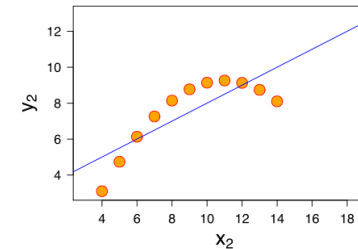
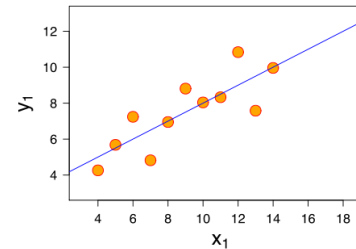
Regression cautions



Plot the data! Regression lines are only appropriate when there is a linear trend in the data

Do not extrapolate too far

Be aware of outliers – they can have an huge effect on the regression line



Linear regression in Python

```
import statsmodels.formula.api as smf
```

```
plt.scatter(x_seq, y_seq)
```

```
lm = smf.ols('y ~ x', data = my_df).fit()
```

```
params = lm.params
```

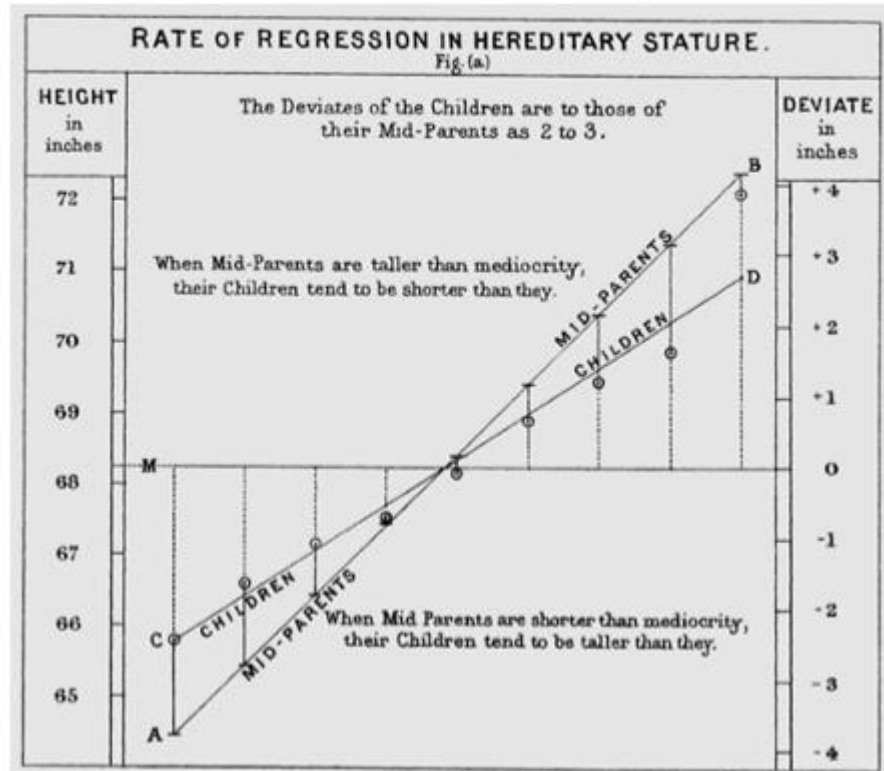
```
sm_predictions = lm.predict(the_data)
```

```
b1, b0 = np.polyfit(x_vals, y_vals, 1) # alternative way
```

Intercept	-526.921684
BA	4744.561329
dtype: float64	

Regression to the mean

Regression to the mean



Original data from Galton, 1885



- Sports Illustrated Cover Jinx
- Rookie of the year curse

Does anyone know what causes this phenomenon?

Performance vs. ability

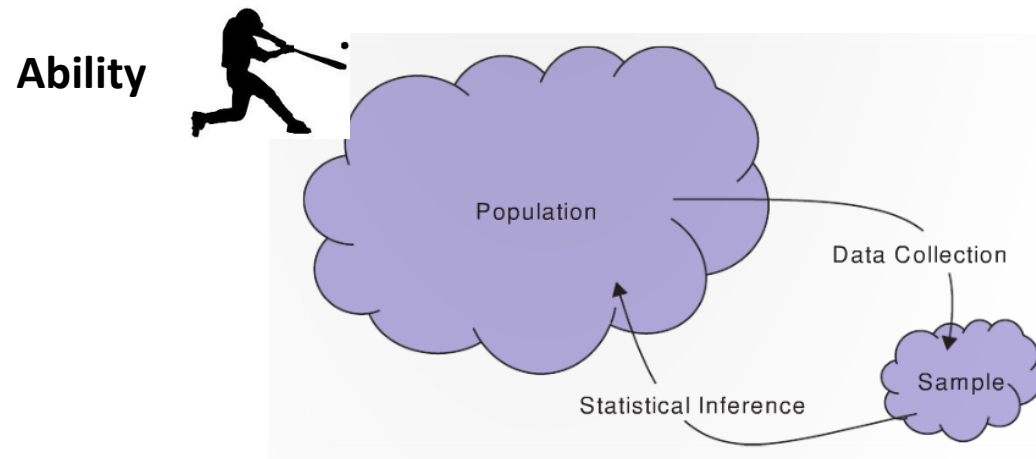
We can define **ability** as a player's innate capabilities

- i.e., if they had infinite plate appearances at their current performance level

We can define **performance** as how well they did in an actual finite set of games

performance = ability + luck

Statistician call this the "error"



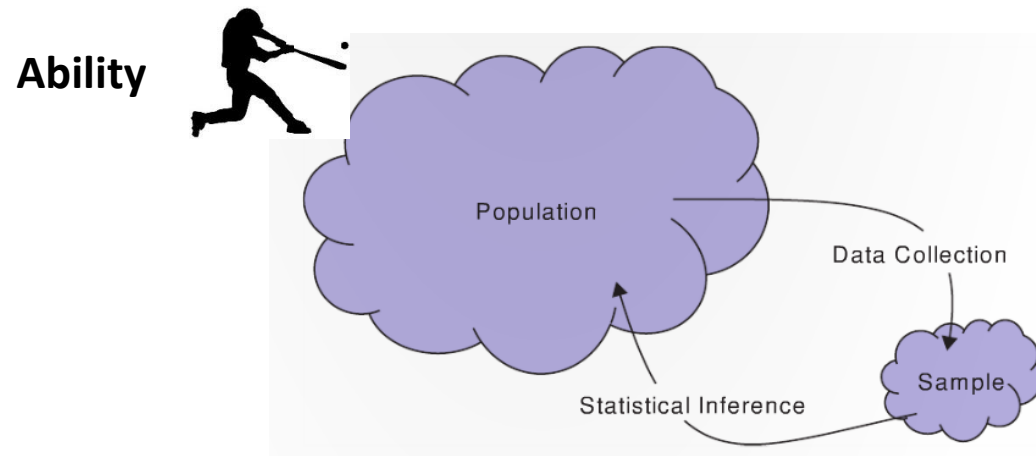
Example: Batting ability vs. performance

Suppose a player had an innate **ability** to get on-base 40% of the time

- i.e., if the player had infinite number of plate appearances they would get on base 40% of the time
- $\pi = 0.400$

If the player only had 600 plate-appearances, the observed OBP could be:

- $\hat{p} = .412$
- $\hat{p} = .382$
- $\hat{p} = .452$
- etc.



Regression to the mean

Since it is impossible to know a player's ability (since they only can play a finite number of games) rookie of the year will be selected based on their performance

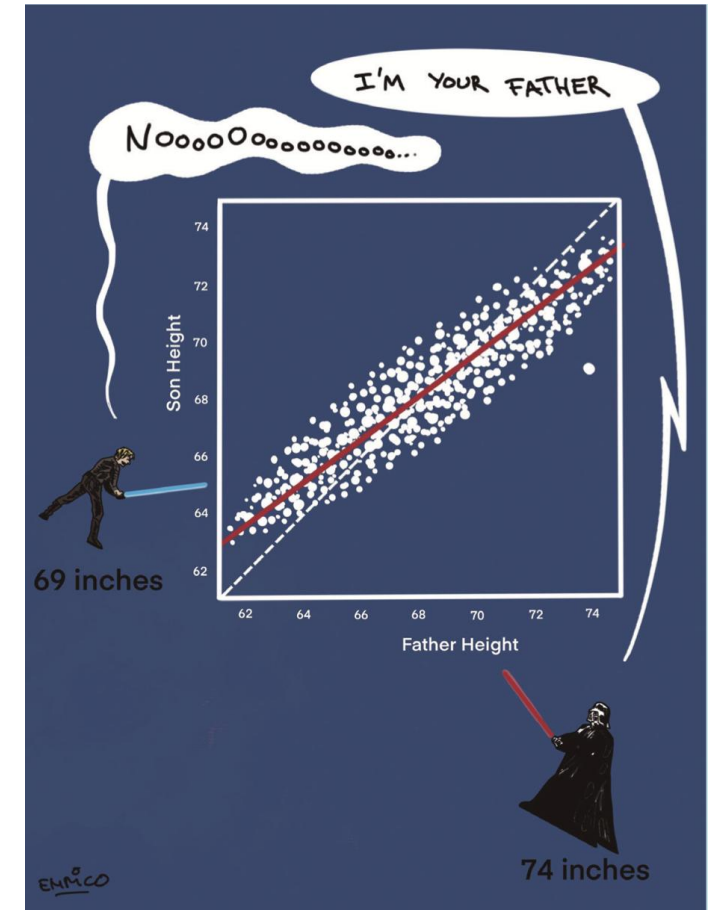
This is likely an over-estimation of their ability

- i.e., selecting based on performance leads "selection bias"

Consequently, their statistics will be closer (i.e., revert) to their real ability in future years

Lab 9 you will explore this in Python!

- Let's quickly explore this now



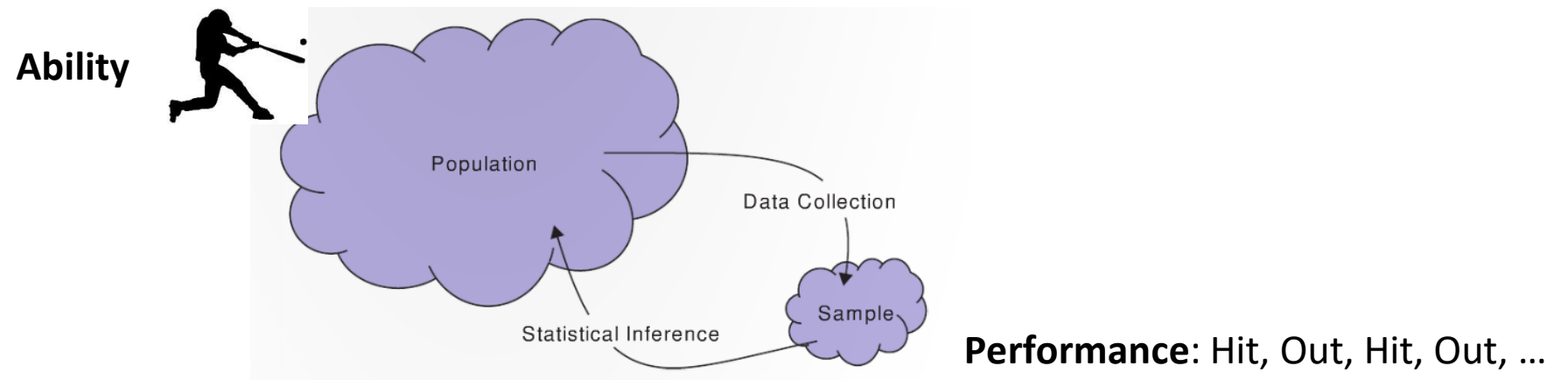
Brief discussion: statistical inference

Statistical inference

Statistical inference: use sample of data to deduce properties of an underlying population or stochastic process

In the context of baseball this usually means: looking at a player's **performance** to tell something about the player's **ability**

- **Ability:** innate talent
- **Performance:** outcomes from playing a number of games



Statistical inference

In this class we have simulating a player's performance based on pre-specified abilities (probability)

- E.g., we can simulate a .333 OBP by rolling a die (or using an Python) to generate random data consistent with a .333 OBP



Hit, Out, Hit, Out, Out, Out, ...

With **statistical inference** we go in the other direction: we take a collection of outcomes and estimate the probability model parameters

Hit, Out, Hit, Out, Out, Out, ...



Estimate π_{hit}



Parameters vs. statistics

A **statistic** is a number that is computed from ***data in a sample***

- i.e., a number summarizing a player's performance
- We denote these with Latin characters ($\hat{p}$, $\bar{x}$, etc.)

A **parameter** is a number that describes some aspect of a ***population or process***

- i.e., a number summarizing a player's ability
- We denote parameters with Greek characters (π , μ , etc.)

Proportions for categorical data

The sample of say size $n = 100$

- out, out, hit, out, walk, out, ..., out

The proportion for a **sample** is a **statistic** denoted by $\hat{p}$

- $\hat{p}_{\text{hit}} = 28/100 = 0.28$

The proportion for a **population/process** is a **parameter** denoted π

- π_{hit} could be the proportion for:
 - Proportion of hits over all plate appearances in a player's career
 - Proportion of future hits a player will have (e.g., a prospect)
 - Proportion of hits if a player batted an infinite number of times

$\hat{p}$ is a **point estimate** of π

- i.e., $\hat{p}$ our best guess of what π is

Statistical inference

Most introductory statistics classes focus on **frequentist** methods

These methods are designed to give the correct answer most of the time

In particular, frequentist methods focus on:

- **Confidence intervals:** give a range of values that will have a parameter in them most of the time
 - 95% of 95% confidence intervals have the parameter in them
- **Hypothesis tests:** Test where a particular parameter value is feasible
 - $H_0: \pi = .400$

Frequentist inference: hypothesis test

Is Shohei Ohtani's batting average ability really .350?

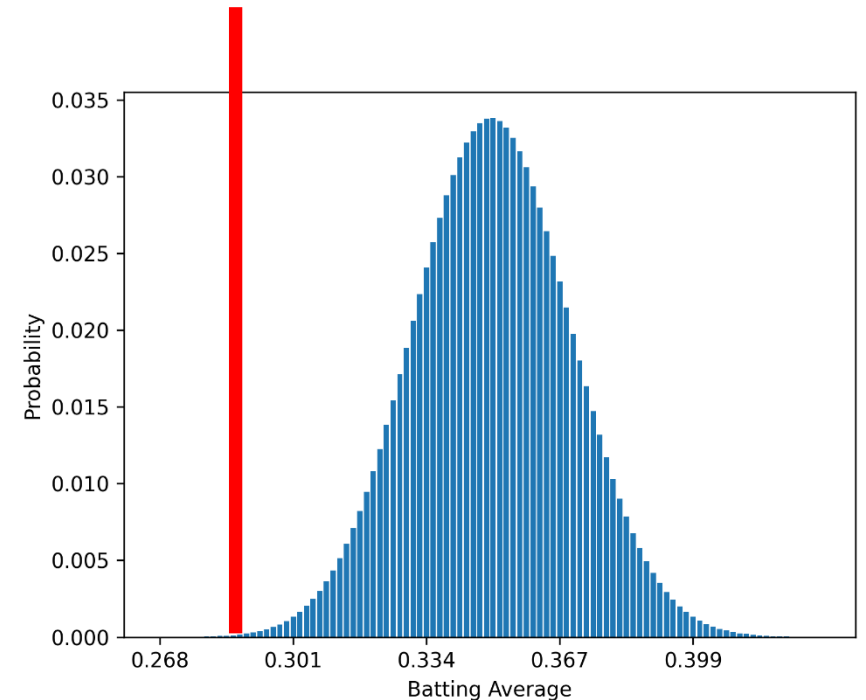
We can write this as:

- $H_0: \pi = .350$
- $H_A: \pi < .350$

We can generate a "null distribution" of statistics ($\hat{p}$'s) consistent with H_0

If Ohtani's observed performance ($\hat{p} = .282$), looks very different from the statistics in the null distribution, we can reject the null hypothesis

- i.e., if the p-value is small we can reject H_0



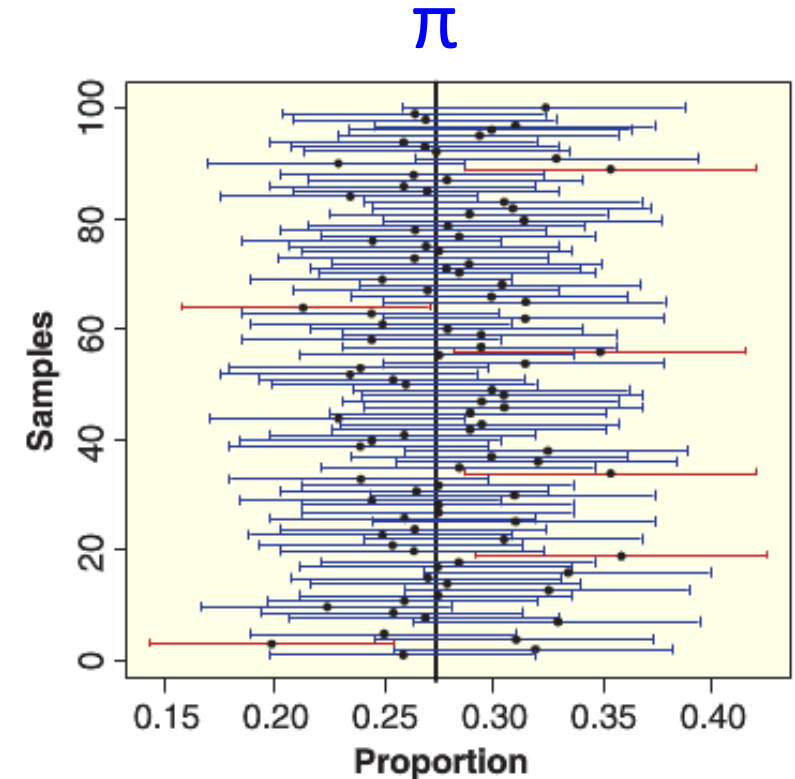
Frequentist inference: Confidence intervals

What is a plausible range of values for Ohtani's batting average?

A **confidence interval** is an interval computed by a method that will contain the *parameter* a specified percent of times

- The **confidence level** is the percent of all intervals that contain the parameter

Confidence interval for Ohtani's batting average ability is: [.246 .318]



$$CI_{95} = \text{statistic} \pm 2 \cdot SE$$

Frequentist inference

Because frequentist inference is discussed in other classes, we will not discuss it much more here

To learn more, take S&DS 1000 and/or many other classes offered at Yale!

Bayesian inference

In Bayesian inference, we estimate the probability that different hypotheses are true

We interpret probability (i.e., a number between 0 and 1) as the degree that we believe in a particular hypothesis

To do Bayesian inference, we combine our prior knowledge/beliefs with the observed data



Probably not a picture of Bayes

$$P(H_i|data) = \frac{P(data|H_i) \cdot P(H_i)}{P(data)}$$

Bayesian Inference: Determining the ability of a player

Suppose there are 3 players with different true OBP abilities:

- Player H_1 's true OBP is .200 $H_1: \pi = .200$
- Player H_2 's true OBP is .333 $H_2: \pi = .333$
- Player H_3 's true OBP is .500 $H_3: \pi = .500$

One player is selected at random, and we observe the player for 10 plate appearances

Can we tell whether it was player H_1 , H_2 , H_3 , who was picked?

Let's simulate this with a 4, 6, and 10 sided die

- 1 or 2 is on base
- Higher numbers are outs
- I will roll the die 10 times...

Bayesian Inference: Determining the ability of a player

Suppose we got 5 on base events out of 10 plate appearances

Question: What die was chosen?

- i.e., what value is π ?

Determining the ability of a player

Here are the simulation results from 1000 simulations of rolling the different dice 10 times:

	0	1	2	3	4	5	6	7	8	9	10
0.200	95	271	315	199	80	35	5	0	0	0	0
0.333	17	81	206	270	226	121	59	16	4	0	0
0.500	2	15	36	108	205	248	196	124	53	21	1

Total number of simulations that produced 5 hits = $35 + 121 + 248 = 404$

$$P(\pi = .200 \mid 5 \text{ hits}) = 35/404 = .087$$

$$P(\pi = .333 \mid 5 \text{ hits}) = 121/404 = .299$$

$$P(\pi = .500 \mid 5 \text{ hits}) = 205/404 = .614$$

Let's quickly explore this in Python!

Multiple regression continued

Multiple regression

In multiple regression we try to predict a quantitative response variable y using several predictor variables $x_1, x_2, \dots, x_k$

For multiple linear regression our equation has the form of:

$$\hat{y} = b_0 + b_1 \cdot x_1 + b_2 \cdot x_2 + \dots + b_k \cdot x_k$$

What are the optimal weights?

$$\text{OPT} = b_1 \cdot \text{BB} + b_2 \cdot \text{HBP} + b_3 \cdot \text{1B} + b_4 \cdot \text{2B} + b_5 \cdot \text{3B} + b_6 \cdot \text{HR} + b_0$$

Let's use multiple regression to find the b_i 's that minimize sum of $(R - \text{OPT})^2$

```
lm = smf.ols('R ~ BB + HBP + H + X2B + X3B + HR', data = teams_2013).fit()
```

```
the_params = lm.params
```

What are the optimal weights?

	b_i
(Intercept)	-497.44
HBP	0.42
BB	0.34
X1B	0.56
X2B	0.75
X3B	1.40
HR	1.44

`lm.params`

Do these coefficients
make sense?

$$\hat{r} = .34 \cdot \text{BB} + .42 \cdot \text{HBP} + .56 \cdot 1\text{B} + .75 \cdot 2\text{B} + 1.40 \cdot 3\text{B} + 1.44 \cdot \text{HR} - 497.44$$

How low can you go?

On lab 8 part 3 you create more complicated regression models to try to get the lowest RMSE

- Whoever came up with the lowest RMSE value wins bragging rights

The winner is...

```
Intercept    -705.028913
BA            -641.294046
SLG          -458.866376
OPS           1809.931871
W              3.138683
SF             0.738574
RA             0.287448
SV            -0.893690
dtype: float64
20.975308372427666
```

Do we think this is a good/best model?



Answer: $\hat{y}_i = -705.0289 + -641.294 \cdot x_{i1} + -458.866 \cdot x_{i2} + 1809.932 \cdot x_{i3} + 3.1386 \cdot x_{i4} + 0.73857 \cdot x_{i5} + 0.2874 \cdot x_{i6} - 0.8937 \cdot x_{i7}$

Non-linear relationships

You can get even lower RMSEs by including non-linear terms

- E.g., $1B^2$, HR^5 etc.

Polynomial regression extends linear regression to non-linear relationships by including nonlinear transformations of predictors

$$BA = \beta_0 + \beta_1 \cdot \text{year} + \beta_2 \cdot (\text{year})^2 + + \beta_3 \cdot (\text{year})^3 + \varepsilon$$

Still a linear equation but non-linear in original predictors

Non-linear relationships

We can add non-linear predictors by simply adding new columns to our table that are non-linear functions of the original columns

```
my_df['x2'] = my_df['x']**2
```

```
lm = smf.ols('y ~ x + x2', my_df).fit()
```

You will also try this on lab 9

Overfitting

Do these optimal weights yield the best model?

As we just discussed, we can use least squares to find the optimal weights:

$$\text{OPT} = w_1 \cdot \text{BB} + w_2 \cdot \text{HBP} + w_3 \cdot \text{1B} + w_4 \cdot \text{2B} + w_5 \cdot \text{3B} + w_6 \cdot \text{HR} + w_0$$

	w_i
(Intercept)	-478.22
HBP	0.52
BB	0.28
X1B	0.52
X2B	0.96
X3B	0.84
HR	1.38

$$\hat{r} = .28 \cdot \text{BB} + .52 \cdot \text{HBP} + .52 \cdot \text{1B} + .96 \cdot \text{2B} + .84 \cdot \text{3B} + 1.38 \cdot \text{HR} - 478.22$$

How good is our new optimal statistic based as measured through RMSE (R^2)?

Is our RSMSE using least squares better than using OPS to predict runs?

OPT* also includes PA
(this is included in OPS too)

Question: do we really believe that OPT* is better at predicting runs than OPS?

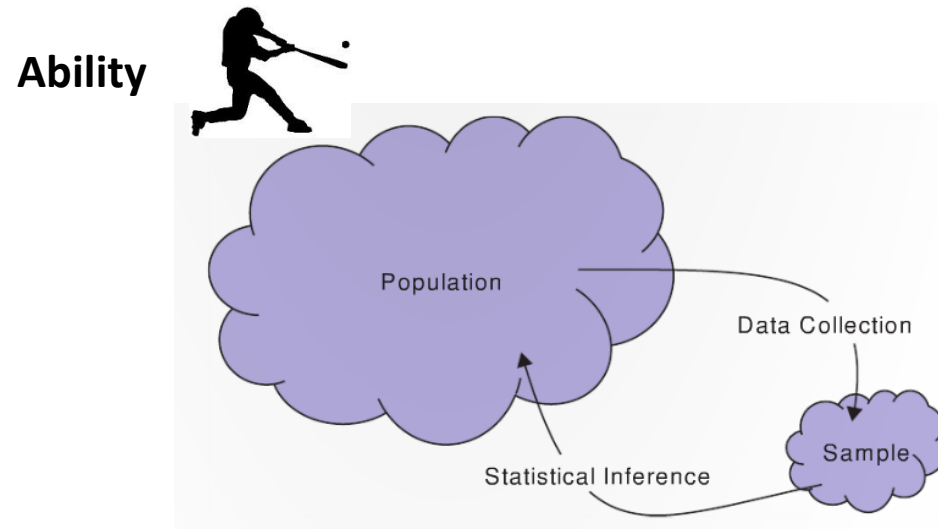
	RMSE
HR	60.42
BA	42.17
OBP	31.83
SlugPct	31.55
OPS	23.46
OPT	24.53
OPT*	21.62

Overfitting

Overfitting occurs when we generate a function that too closely matches random sample we have, but does not generalize to the full probability distribution

[Overfitting song...](#)

- The model is fit too closely to observed performance and not getting at the players' ability

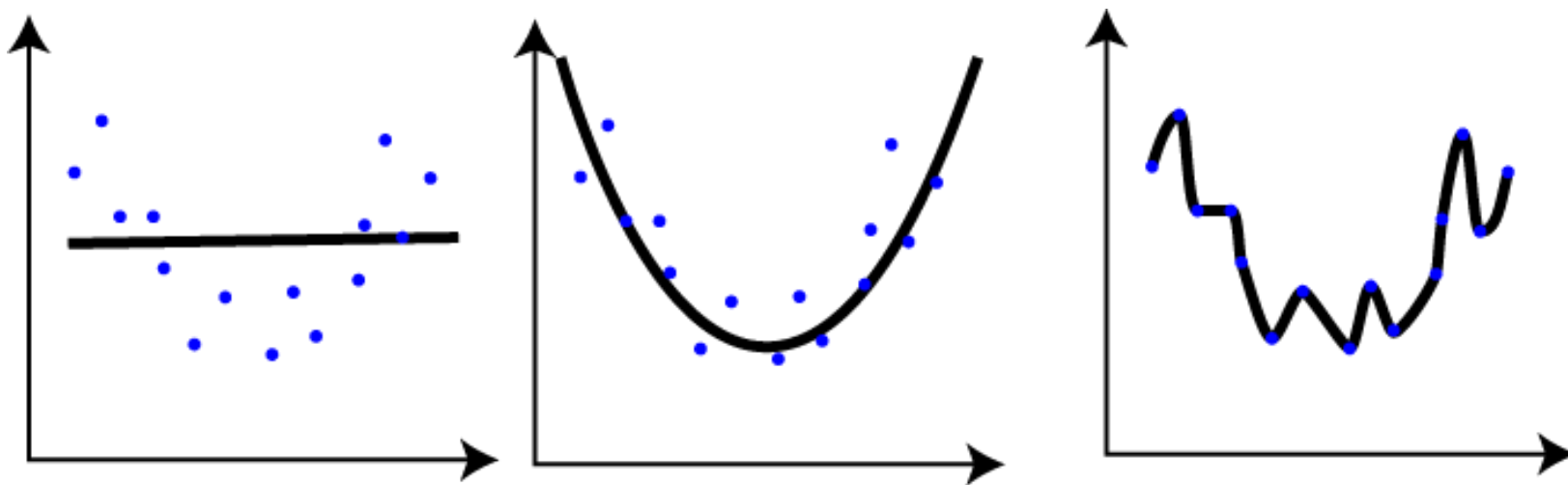


Fitting on the 2012 season,
measuring the fit on the 2013 season

	RMSE
HR	62.85
BA	49.29
OBP	38.40
Slug	34.50
OPS	26.61
OPT*	30.39

“Optimal” fit no longer that optimal

Overfitting



Cross-validation

To realistically assess how well our classifier can make accurate predictions on new data (i.e. to estimate the generalization error) we use cross-validation

Cross-validation consists of splitting your data into two sets

A training set in which the parameters of classification/regression model are fit

A test set in which the prediction accuracy of our model is assessed



Cross-validation

Training error rate: model predictions are made on using the same data that the model was fit with

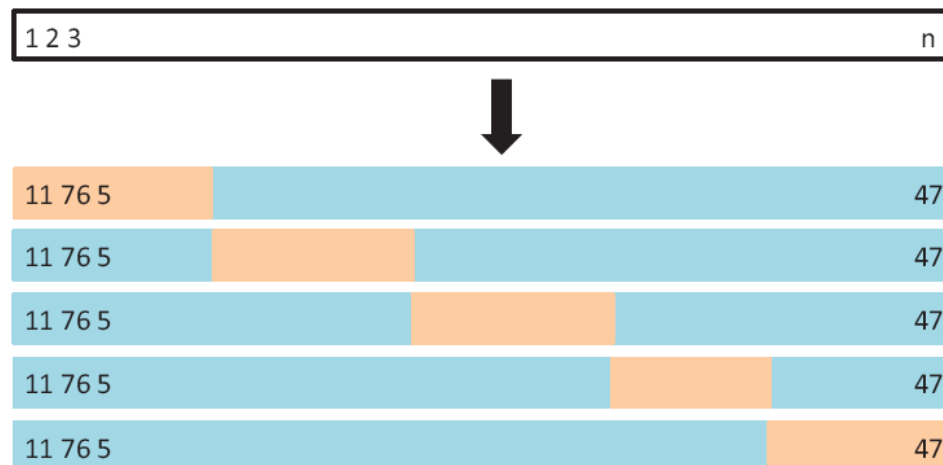
Test error rate: model predictions are made on a separate set of data

The test error rate is an estimate of how accurate your predictions will be on new (future) data

K-fold cross-validation

K-fold cross-validation

- Split the data into k parts
- Train on $k-1$ of these parts and test on the left out part
- Repeat this process for all k parts
- Average the prediction accuracies to get a final estimate of the generalization error



Leave-one-out (LOO)
cross-validation: $k = n$

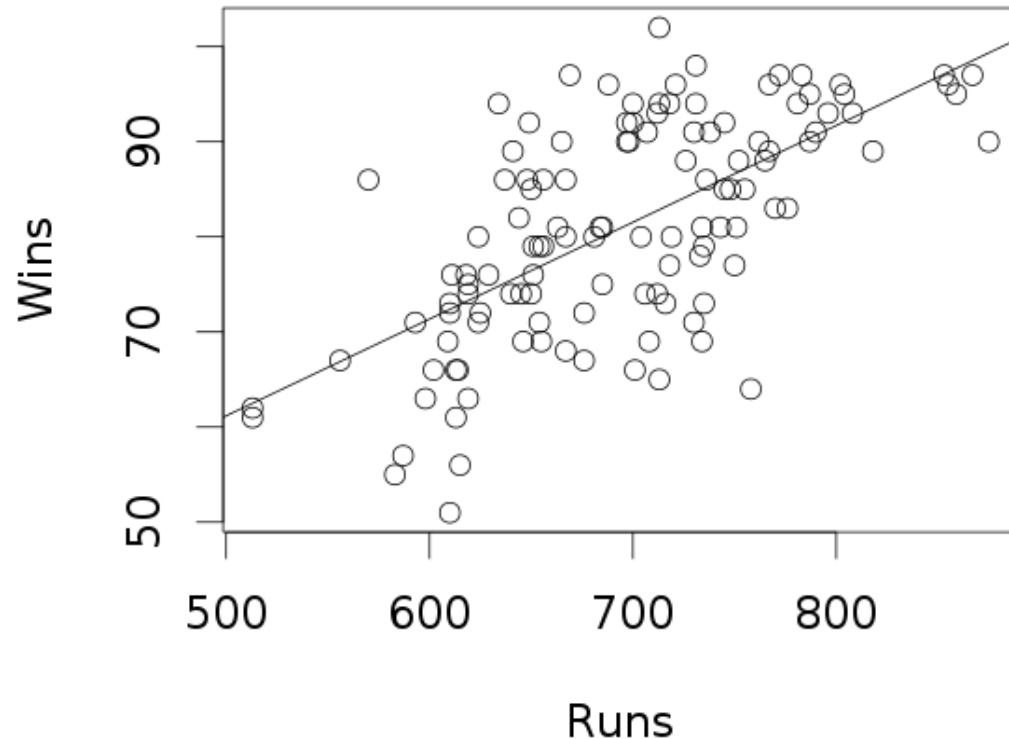
Fitting on the 2012 season,
measuring the fit on the 2013 season

	RMSE
HR	62.85
BA	49.29
OBP	38.40
Slug	34.50
OPS	26.61
OPT*	30.39

This is a form of cross-validation! (out of sample predictions)

Bill James' "Pythagorean Method"

Our linear equation for predicting the number of **wins** a team would score as a **function of the number of runs** they produced had some issues...



$$\hat{w} = 14.47 + .088 \cdot \text{Runs}$$

What happens when 0 runs are scored all season?

Bill James' "Pythagorean Method"

Bill James came up with a formula that he called the "Pythagorean Method" that relates:

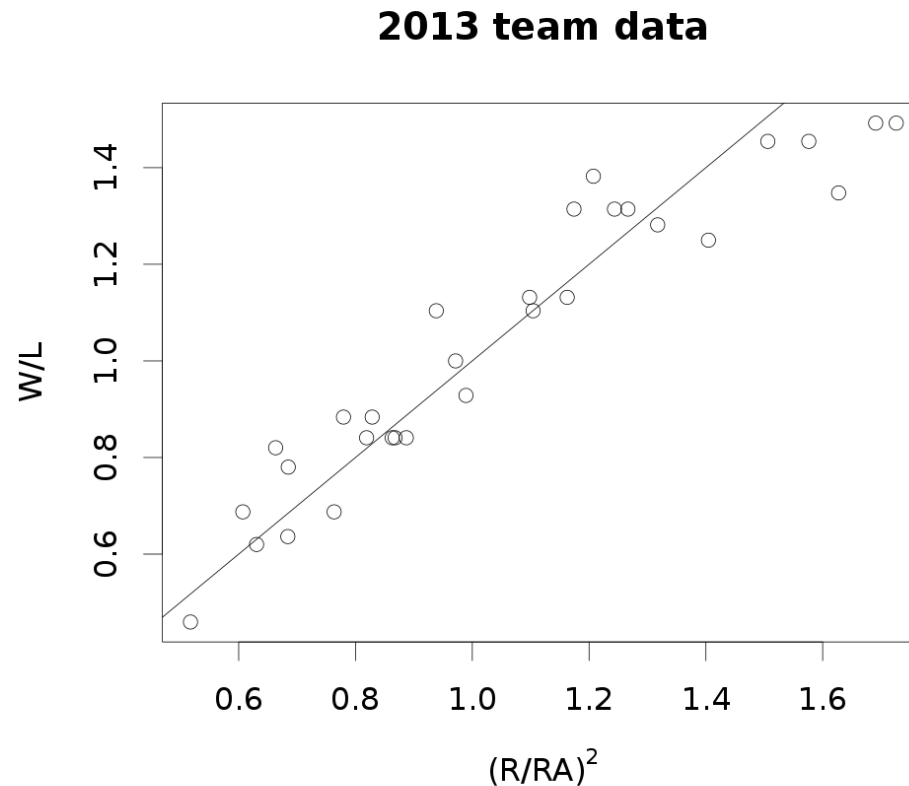
- wins (W) and losses (L) to
- runs scored (R) and runs allowed (RA)

$$\frac{W}{L} = \left(\frac{R}{RA} \right)^2$$

What happens when a team scores 0 runs with this formula?

How can we tell how good this formula is?

An answer: look at a scatter plot of W/L ratio predicted by $(R/RA)^2$ and the actual W/L ratio



How can we tell how good this formula is?

An answer: compare the number of wins predicted by the R and RA values, to the number of wins actually scored by each team

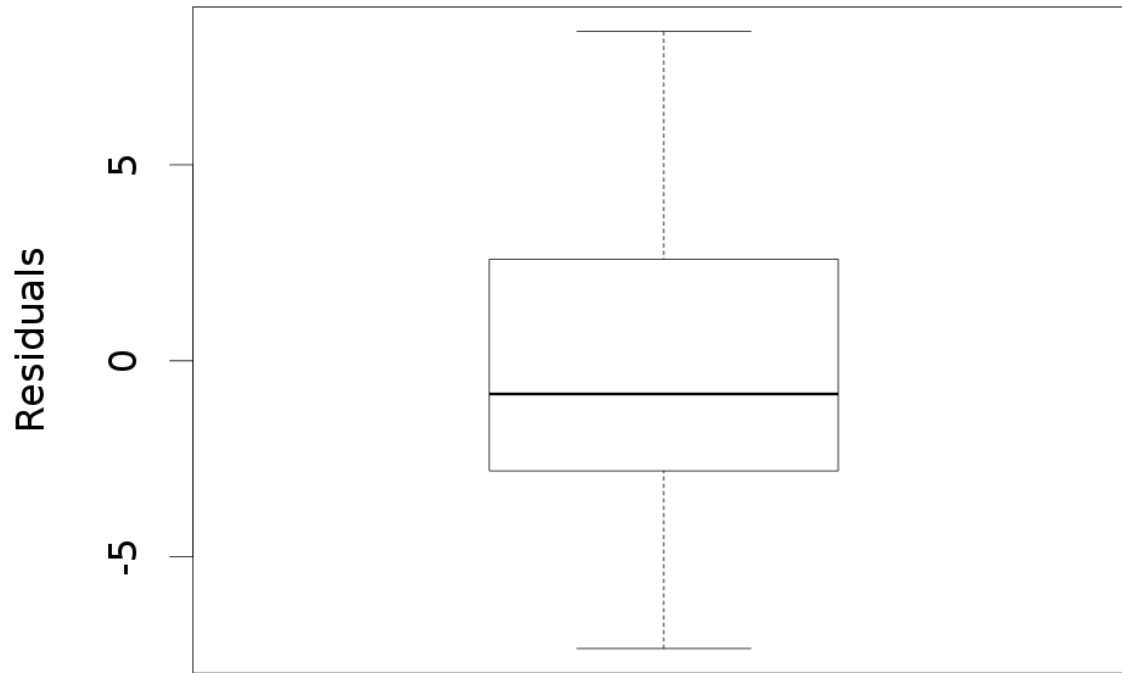
- i.e., look at the residuals of $W - \hat{W}_{\text{Pythag}}$

$$(W/L)_{\text{pred}} = (R/RA)^2$$

$$(W_{\text{pred}} / (162 - W_{\text{pred}})) = (R/RA)^2 \quad \dots \text{ some algebra } \dots$$

$$W_{\text{pred}} = (162 \cdot (R/RA)^2) / (1 + (R/RA)^2)$$

How can we tell how good this formula is?



RMSE = 3.9

95% of the time off by < 8 wins

- Assuming the residuals are normal

Five number summary of the residuals:
(-7.34, -2.81, -0.85, 2.59, 8.30)

Can we do better the James' formula?

Any ideas how we could modify James' formula to do better?

One idea: try to find a better exponent on R/RA rather than just assuming it is 2

$$\frac{W}{L} = \left(\frac{R}{RA} \right)^2 \qquad \frac{W}{L} = \left(\frac{R}{RA} \right)^k$$

How can we do this?

Can we do better the James' formula?

If we take the logarithm of James' formula, it becomes a linear equation

$$\log\left(\frac{W}{L}\right) = 2 \cdot \log\left(\frac{R}{RA}\right)$$

$$\log\left(\frac{W}{L}\right) = k \cdot \log\left(\frac{R}{RA}\right)$$

Since this equation is linear we can find k with linear regression!

Can we do better the James' formula?

In R:

```
lm(formula = log(W_L_ratio) ~ log(R_RA_ratio) -1)
```

Coefficients:

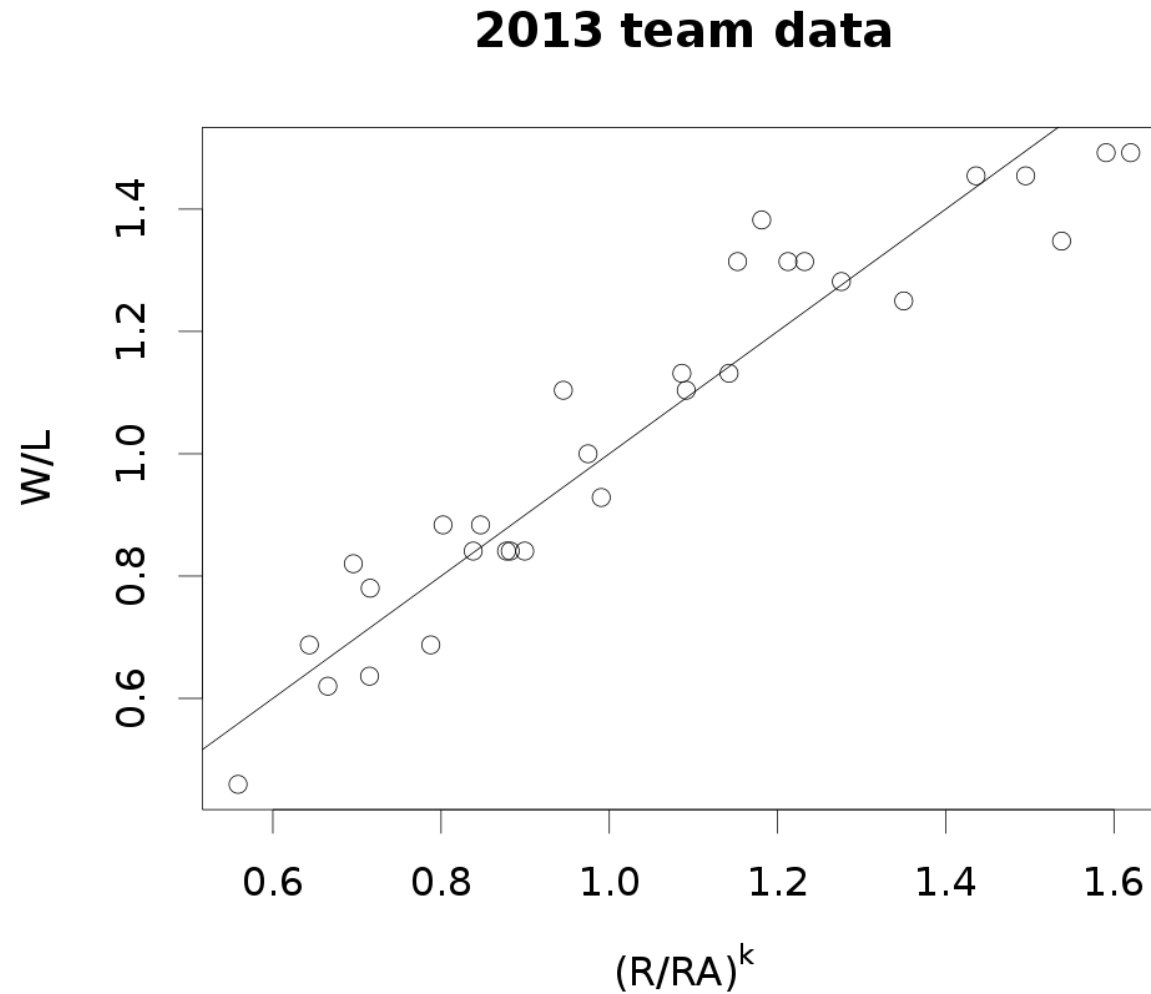
```
log(R_RA_ratio)
```

```
1.7675268
```

$$\frac{W}{L} = \left(\frac{R}{RA} \right)^{1.77}$$

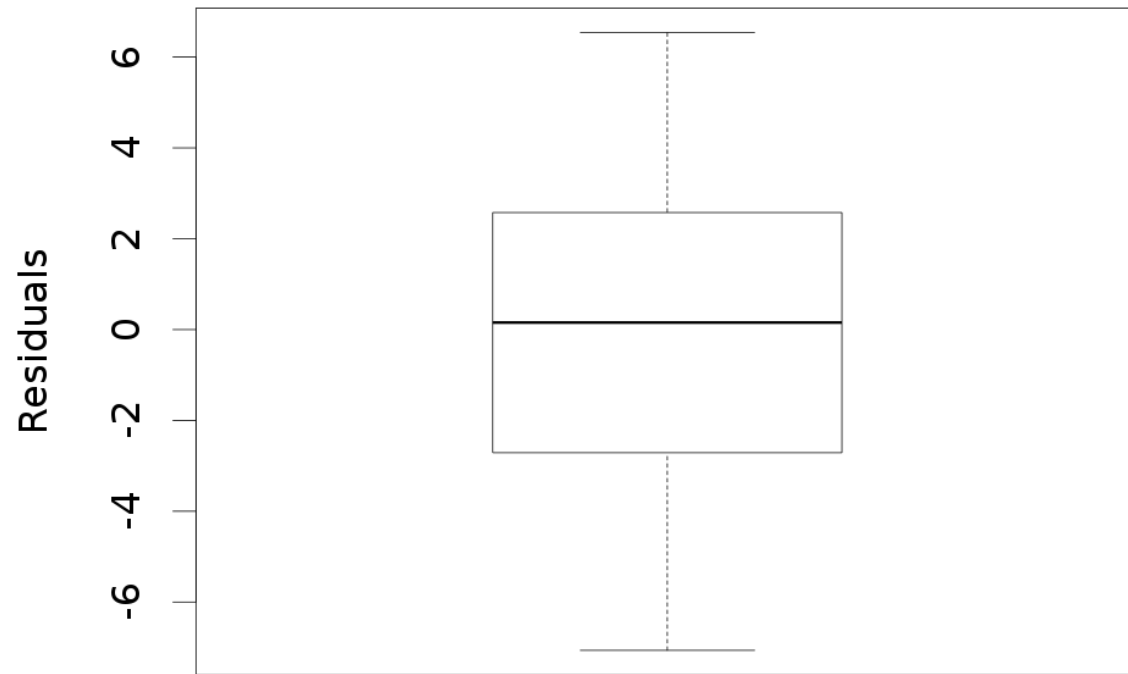
You will try this in Python on a slightly different data set for homework 9!

Can we do better the James' formula?



Where $k = 1.77$

Can we do better the James' formula?



Five number summary of the residuals:

Old: (-7.34, -2.81, -0.85, 2.59, 8.30)

New: (-7.06, -2.71, 0.15, 2.57, 6.54)

Old: RMSE = 3.9

New: RMSE = 3.6

Is this an improvement?

How can we better assess if this is a real improvement?

- Cross-validation!

Lab 9

If there's time, we can start on lab 9 now

Or you can work on your project